UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Web Applications A.Y. 2025-2026

Homework 1 – Server-side Design and Development

Master Degree in Computer Engineering
Master Degree in Cybersecurity
Master Degree in ICT for Internet and Multimedia

Group Acronym	KLZWAP	
Last Name	**First Name**	**Badge Number**
Dusi	Luca	2200074
Kosa	Laszlo	2183964
Trifunovic	Milos	2183388
Zanella	Edoardo	2195316

1 Objectives

The goal of this project is to design and develop a server-side Web application called **Lecture Notes App**: a platform that allows university students to upload, share, search, and download lecture notes in PDF format. Users can register and log in to the platform, organize notes by course, mark notes as favourites, and rate content shared by other students.

The main objectives of this project are:

- Provide a platform for sharing lecture notes
- Enable efficient search and organization of content
- Allow user interaction through ratings and favourites
- Ensure scalability and maintainability of the system

1.1 Technologies Used

The application is developed using:

- Java Servlets and JSP for server-side logic and dynamic content generation
- PostgreSQL as the relational database management system
- Apache Tomcat as the deployment server

1.2 Motivation

University students often struggle to find well-organized and reliable lecture notes. Existing solutions are either fragmented or lack quality control. This project aims to provide a centralized and user-driven platform to improve accessibility and collaboration.

2 Main Functionalities

This section describes the main functionalities offered by the application. The system is designed to manage user accounts, handle lecture notes, and provide interaction mechanisms between users.

2.1 User Management

The application allows users to create and manage their personal accounts. In particular, users can register, log in, log out, and update their profile information. These features are implemented through the following components:

- `RegisterServlet`
- `LoginServlet`
- `LogoutServlet`
- `ProfileServlet`

2.2 Note Management

Authenticated users can upload and manage lecture notes in PDF format. Each note is associated with a specific course and can be modified or removed by its owner. The system also allows users to download notes shared by others.

- `UploadNoteServlet`
- `UpdateNoteServlet`
- `DeleteNoteServlet`
- `DownloadNoteServlet`

2.3 Search and Organization

To facilitate access to content, the application provides a search functionality that allows users to find notes based on different criteria such as title, course, or author.

- `SearchNoteServlet`

Additionally, notes are organized by course, and a summary view is available to display all notes related to a specific course.

- `CourseSummaryServlet`

2.4 User Interaction Features

The platform includes features that enhance user interaction and content evaluation.

- **Favourites:** users can save notes in a personal favourites list and access them through a dedicated page (`favorites.jsp`).
- **Ratings:** users can rate notes uploaded by other students, contributing to an overall quality score.

3 Data Logic Layer

After presenting the main functionalities of the application, this section focuses on the data logic layer, which is responsible for modeling and managing the information handled by the system. In particular, the structure of the database and the relationships between entities are described through an Entity-Relationship schema.

3.1 Entity-Relationship Schema

The database is designed to represent the core concepts of the application through a set of entities and relationships. The main entities are: *User*, *Note*, and *Course*. The system also includes relationships between these entities, namely *Uploads*, *Favorite*, *Rating*, and *Belongs to*. In particular, *Favorite* and *Rating* are associative relationships that model many-to-many interactions between users and notes. The relationships between these elements can be summarized as follows:

- A User can upload multiple Notes (1:N relationship)
- A User can mark multiple Notes as Favorite (M:N relationship)

- A User can assign at most one Rating to each Note, while being able to rate multiple Notes (M:N relationship with constraint of uniqueness)
- Each Note belongs to exactly one Course (N:1 relationship)

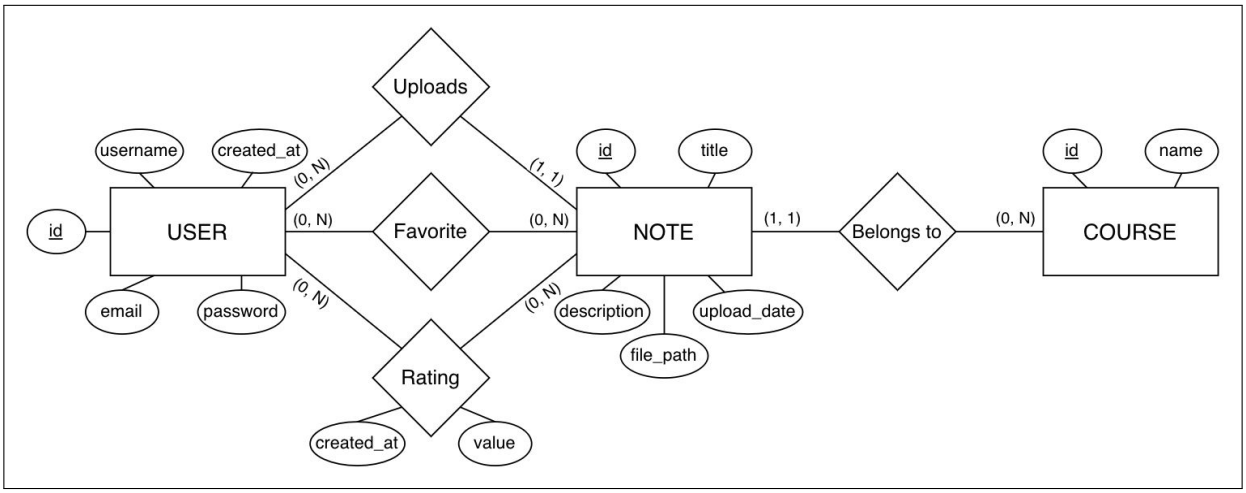

Figure 1: Entity-Relationship Schema of the Lecture Notes App

The ER diagram in Figure 1 provides a visual representation of the database structure and highlights these relationships.

3.2 Entity and Relationship Description

This section describes both the entities and relationships of the system, along with their mapping to the relational database schema.

User The *User* entity stores authentication credentials and profile information, including: `id`, `username`, `email`, `password`, and `createdAt`.

Course The *Course* entity represents a subject or course and includes: `id` and `name`.

Note The *Note* entity contains the main information related to uploaded lecture notes, including: `id`, `author_id`, `course_id`, `title`, `description`, `upload_date`, and `file_path`. From the relational perspective:

- The `author_id` attribute is a foreign key referencing `User(id)`, implementing the *Uploads* relationship.
- The `course_id` attribute is a foreign key referencing `Course(id)`, implementing the *Belongs to* relationship.

These relationships are therefore represented through foreign keys and do not require separate tables.

Uploads (Relationship) The *Uploads* relationship models the association between a User and the Notes they upload. It is a one-to-many (1:N) relationship and is implemented in the relational schema through the foreign key `author_id` in the *Note* table. As a result, no separate table is required.

Favorite (Relationship) The *Favorite* relationship models the association between users and their favorite notes. Since it is a many-to-many relationship, it is mapped to a separate relational table including: `user_id`, `note_id` and `created_at`.

Rating (Relationship) The *Rating* relationship represents the evaluation given by a user to a note. This many-to-many relationship is also mapped to a separate table including: `id`, `user_id`, `note_id`, `value`, and `created_at`. A uniqueness constraint on (`user_id`, `note_id`) ensures that each user can assign only one rating per note.

Belongs to (Relationship) The *Belongs to* relationship associates each *Note* with a specific *Course*. It is a many-to-one (N:1) relationship and is implemented through the foreign key `course_id` in the *Note* table. This relationship is therefore not mapped to a separate table.

4 Presentation Logic Layer

After defining the data model of the application, this section describes the presentation layer, which is responsible for user interaction through the web interface. The presentation layer is implemented using JSP pages located in `src/main/webapp/jsp/`. The application entry point is `login.jsp`, as specified in the `web.xml` configuration file.

4.1 Login Page – `login.jsp`

Presents a form for user authentication (username and password). On success the user is redirected to the home page.

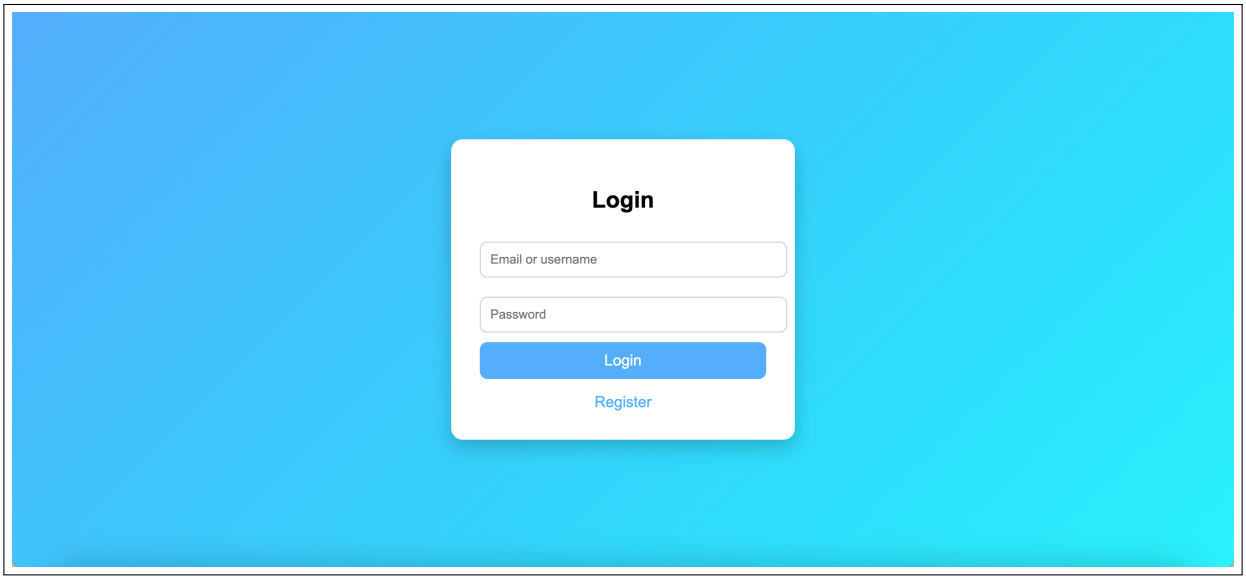

Figure 2: Login page

4.2 Register Page – `register.jsp`

Allows new users to create an account by providing username, email, and password.

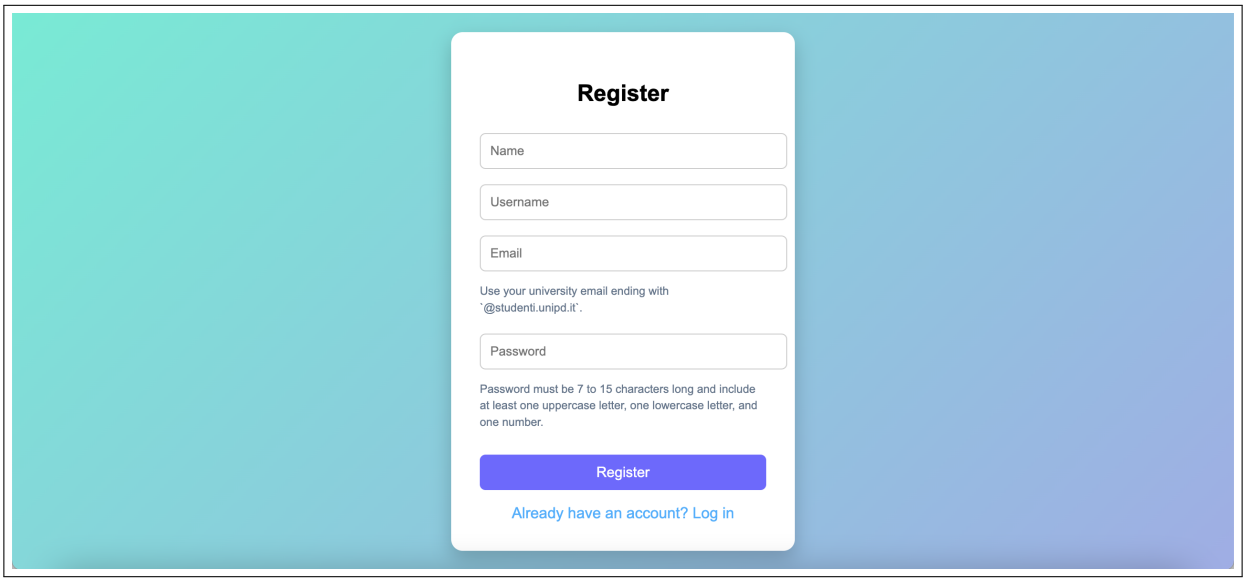

Figure 3: Registration page

4.3 Home Page – `home.jsp`

The main dashboard after login. Displays the list of available notes with search and filter controls. Users can browse by course, download notes, or add them to favourites directly from this page.

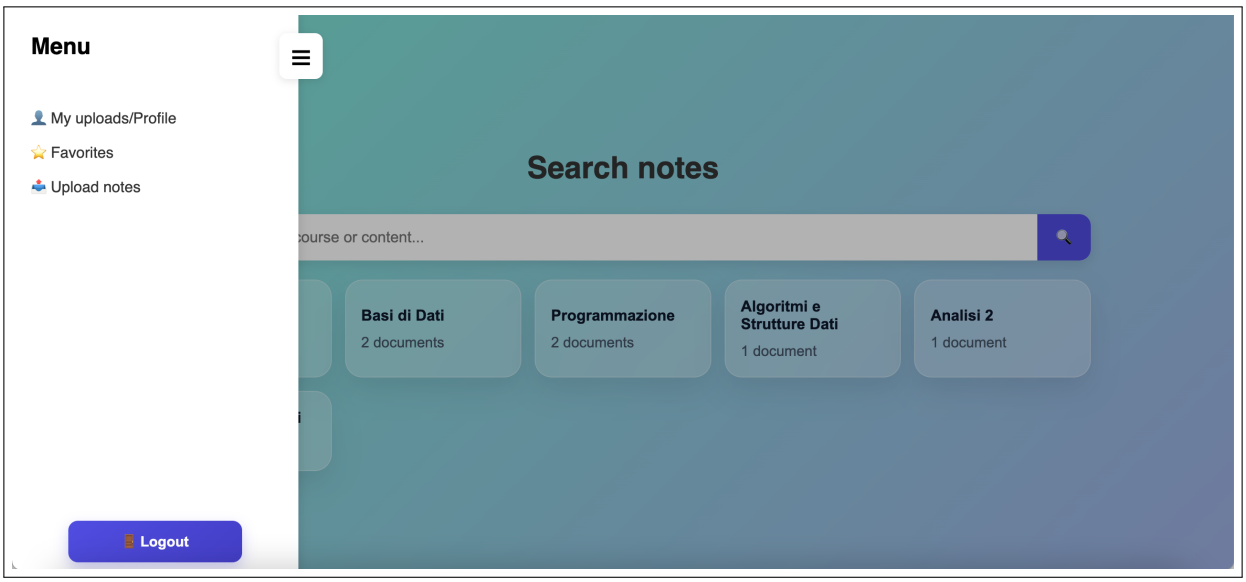

Figure 4: Home page

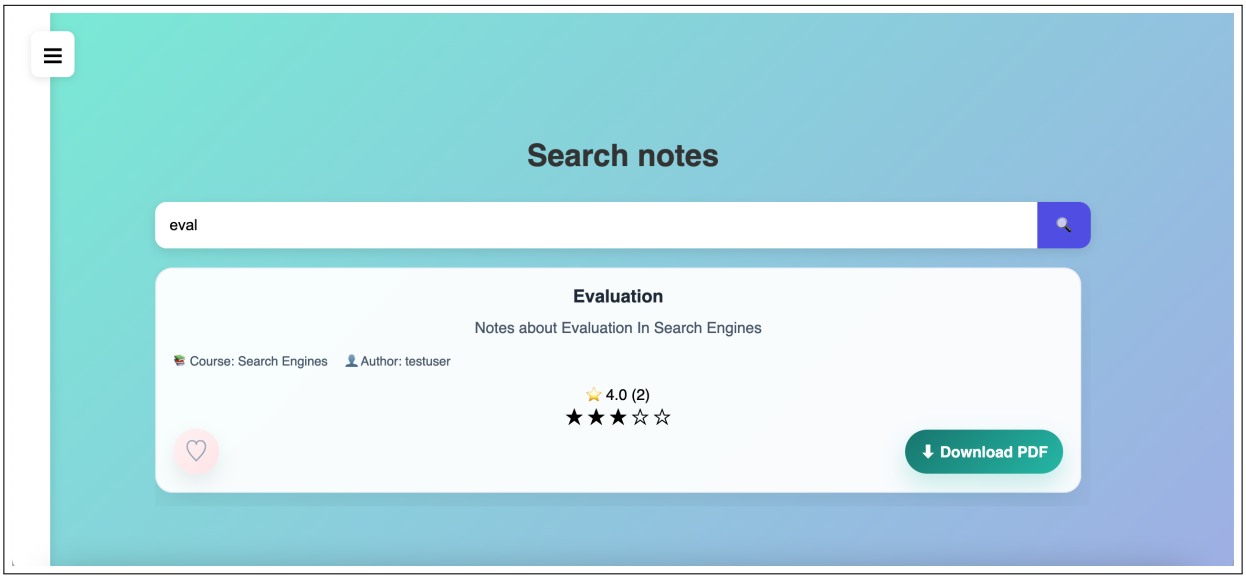

Figure 5: Search note

4.4 Upload Page – `upload.jsp`

Form for uploading a new note: the user selects the course, inserts a title and description, and attaches a PDF file.

Figure 6: Upload page

4.5 Profile Page – `profile.jsp`

Shows the user's personal information and the notes they have uploaded. Allows updating profile data and deleting own notes.

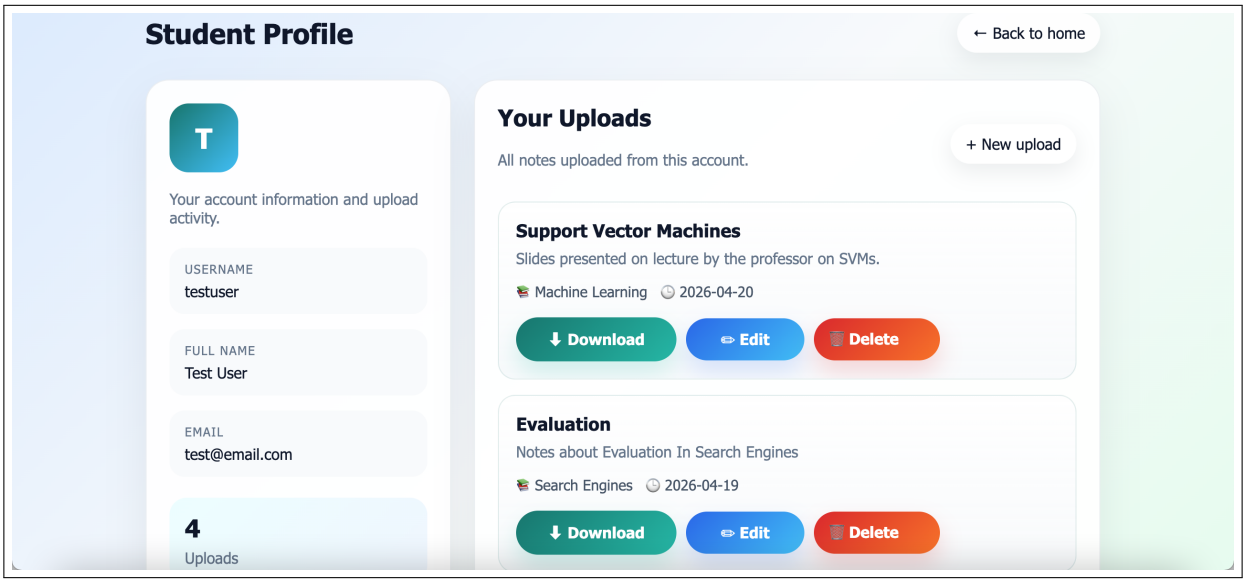

Figure 7: Profile page

4.6 Favourites Page – `favorites.jsp`

Lists all notes that the logged-in user has marked as favourite.

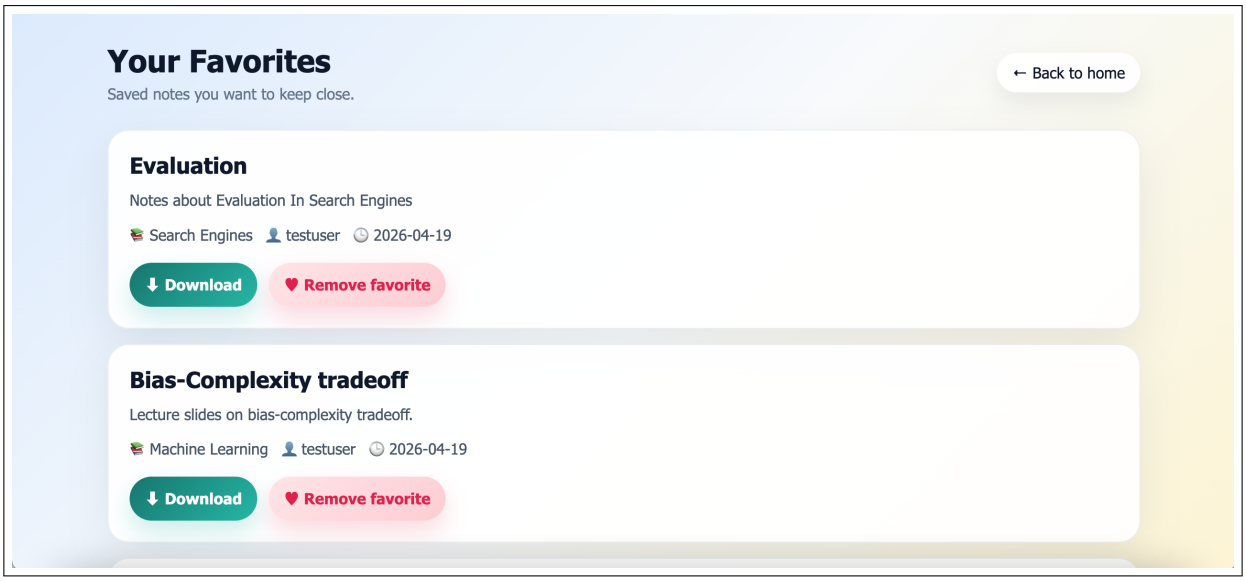

Figure 8: Favourites page

Each page is designed to provide a simple and intuitive user experience, ensuring easy navigation and efficient interaction with the system.

5 Business Logic Layer

5.1 Class Diagram

The class diagram shown in Figure 9 provides an overview of the main components of the system and their relationships.

The architecture is structured into the following layers:

- **Controller Layer** – implemented through Java Servlets, responsible for handling HTTP requests and responses. Examples include `UploadNoteServlet`, `LoginServlet`, and `SearchNoteServlet`.
- **Model Layer** – composed of Plain Java Objects representing the core entities of the application, such as `User`, `Note`, `Course`, `Favorite`, and `Rating`.
- **DAO Layer** – responsible for data persistence and retrieval through JDBC interaction with the PostgreSQL database. Examples include `NoteDAO`, `UserDAO`, and `RatingDAO`.
- **Utility / Service Layer** – provides supporting functionalities such as database connection management and file storage. In particular, `DBConnection` handles database connectivity, while `StorageService` manages file upload and retrieval from cloud storage.

This layered design promotes loose coupling between components and simplifies future extensions of the system.

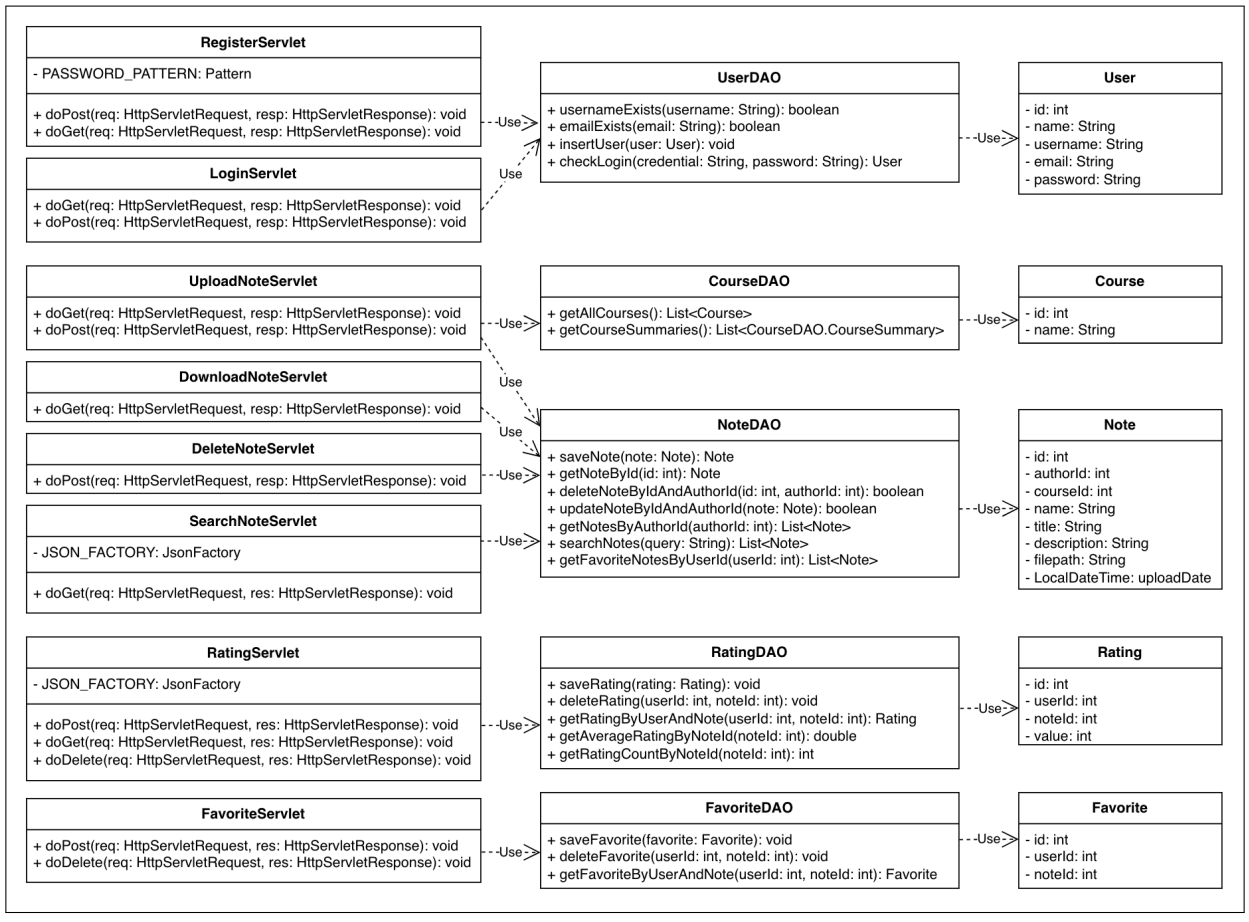

Figure 9: Class Diagram of the Lecture Notes App

5.2 Sequence Diagram

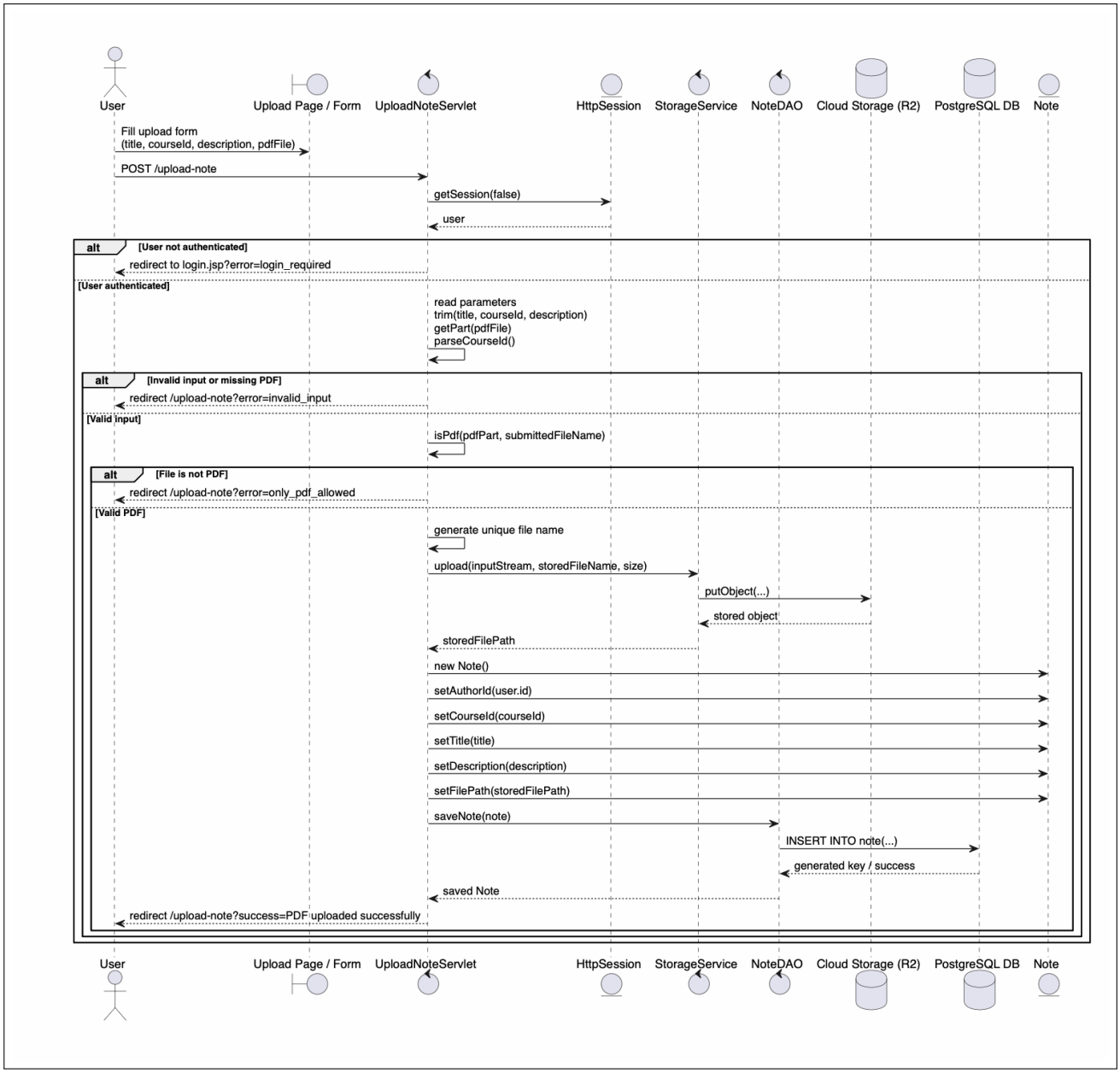

Figure 10: Sequence Diagram – Note Upload Flow

A representative interaction of the system is illustrated through the note upload process, which is one of the core functionalities of the application. The sequence diagram in Figure 10 shows how the different components collaborate during this operation. When the user submits the upload form, a `POST multipart/form-data` request is sent to the `/upload-note` endpoint. The `UploadNoteServlet` handles the request by first verifying the existence of a valid user session. If the user is not authenticated, the request is rejected and the user is redirected to the login page. If the session is valid, the servlet extracts and validates the input parameters, including the note metadata and the uploaded file. The system ensures that all required fields are provided and that the file is in

PDF format. Once validated, a unique file name is generated and the file is uploaded to cloud storage through the `StorageService`. The service interacts with the storage backend to persist the file and returns the file path. Subsequently, the servlet creates a `Note` object containing all relevant metadata, such as author, course, title, description, and file path. This object is then passed to `NoteDAO`, which stores the information in the PostgreSQL database. Finally, depending on the outcome of the operation, the user is redirected either to a success page or to an error page. This sequence diagram effectively highlights the interaction between presentation, business logic, and data layers, providing a clear view of the system workflow.

5.3 REST API Summary

URI	Method	Description	Filter
`/register`	GET	Loads the registration page	No
`/register`	POST	Register a new user after validation	No
`/login`	GET	Loads the login page	No
`/login`	POST	Authenticate user and create session	No
`/logout`	POST	Invalidate session and redirect to login	Yes
`/logout`	GET	Perform the same logout logic as POST	Yes
`/profile`	GET	Load the logged-in user's profile page	Yes
`/rest/favorites/{noteId}`	POST	Add a note to the logged-in user's favorites	Yes
`/rest/favorites/{noteId}`	DELETE	Remove a note from the logged-in user's favorites	Yes
`/favorites`	GET	Loads the favorites page with the user's favorite notes	Yes
`/remove-favorite`	POST	Remove a favorite note and redirect to favorites page	Yes
`/delete-note`	POST	Delete an uploaded note, related ratings, and file	Yes
`/download-note`	GET	Download the file attached to a note	Yes
`/rest/notes/search`	GET	Search notes by query and return matching notes as JSON	Yes
`/update-note`	POST	Update a logged-in user's uploaded note	Yes
`/upload-note`	GET	Loads the upload page	Yes
`/upload-note`	POST	Upload a PDF note and save its metadata	Yes
`/rest/ratings/{noteId}`	POST	Create or update the logged-in user's rating	Yes
`/rest/ratings/{noteId}`	DELETE	Delete the logged-in user's rating	Yes
`/rest/ratings/{noteId}`	GET	Return rating details for a note as JSON	Yes
`/rest/courses/summary`	GET	Returns course summaries in JSON	Yes

Table 2: REST API summary

5.4 REST Error Codes

Error Code	HTTP Status Code	Description
DATABASE_ERROR	500 Internal Server Error	Generic database failure returned when a DAO operation throws an SQLException.
USER_NOT_AUTHENTICATED	401 Unauthorized	Returned when the client tries to create or delete favorites or ratings without an authenticated user session.
MISSING_QUERY	400 Bad Request	Returned by the note search endpoint when the query parameter is missing or blank.
MISSING_NOTE_ID	400 Bad Request	Returned by rating creation or deletion when the note identifier is missing from the request path.
NOTE_ID_MISSING_IN_PATH	400 Bad Request	Returned by rating retrieval when the note identifier is missing from the request path.
INVALID_NOTE_ID	400 Bad Request	Returned when the provided note identifier is not valid, for example because it is non-numeric or not positive.
INVALID_PARAMETERS	400 Bad Request	Returned by rating creation or deletion when request parameters cannot be parsed correctly.
INVALID_VALUE	400 Bad Request	Returned when a rating value is outside the accepted range from 1 to 5.

Table 3: REST API error codes

5.5 REST API Details

User Registration

- URL: /register
- Method: GET / POST
- URL Parameters: None
- Data Parameters: GET – none; POST – username (string), email (string), password (string)
- Success Response: GET – 200 OK, forwards to register.jsp; POST – 302 Found, redirects to login page
- Error Response: 400 Bad Request if required fields are missing or invalid; 409 Conflict if username or email already exists; 500 Internal Server Error on database failure

User Authentication

- URL: /login
- Method: GET / POST
- URL Parameters: None
- Data Parameters: GET – none; POST – username (string), password (string)
- Success Response: GET – 200 OK, forwards to login.jsp; POST – 302 Found, redirects to home page with session created
- Error Response: 401 Unauthorized if credentials are invalid; 500 Internal Server Error on database failure

User Logout

- URL: `/logout`
- Method: `GET` / `POST`
- URL Parameters: None
- Data Parameters: None
- Success Response: `302 Found` – session invalidated, redirects to login page
- Error Response: `401 Unauthorized` if no active session exists

User Profile

- URL: `/profile`
- Method: `GET`
- URL Parameters: None
- Data Parameters: None
- Success Response: `200 OK` – forwards to `profile.jsp` with the logged-in user's data and uploaded notes
- Error Response: `401 Unauthorized` if the user is not authenticated; `500 Internal Server Error` on database failure

Note Upload

- URL: `/upload-note`
- Method: `GET` / `POST`
- URL Parameters: None
- Data Parameters: `GET` – none; `POST` – `title` (string), `description` (string), `courseId` (int), `file` (PDF, multipart/form-data)
- Success Response: `GET` – `200 OK`, forwards to `upload.jsp`; `POST` – `302 Found`, redirects to home page
- Error Response: `400 Bad Request` if required fields are missing or the file is not a PDF; `401 Unauthorized` if the user is not authenticated; `500 Internal Server Error` on storage or database failure

Note Download

- URL: `/download-note`
- Method: `GET`
- URL Parameters: `noteId` (int, required)
- Data Parameters: None
- Success Response: `200 OK` – streams the PDF file with `Content-Type: application/pdf`
- Error Response: `400 Bad Request` if `noteId` is missing or invalid; `401 Unauthorized` if the user is not authenticated; `404 Not Found` if the note does not exist; `500 Internal Server Error` on storage failure

Note Update

- URL: `/update-note`
- Method: `POST`
- URL Parameters: None
- Data Parameters: `noteId` (int), `title` (string), `description` (string), `courseId` (int)
- Success Response: `302 Found` – redirects to profile page
- Error Response: `400 Bad Request` if required fields are missing; `401 Unauthorized` if the user is not authenticated; `403 Forbidden` if the requesting user is not the note author; `500 Internal Server Error` on database failure

Note Deletion

- URL: `/delete-note`
- Method: `POST`
- URL Parameters: None
- Data Parameters: `noteId` (int)
- Success Response: `302 Found` – redirects to profile page
- Error Response: `401 Unauthorized` if the user is not authenticated; `403 Forbidden` if the requesting user is not the note author; `500 Internal Server Error` on database or storage failure

Note Search

- URL: `/rest/notes/search`
- Method: `GET`
- URL Parameters: `query` (string, required)
- Data Parameters: None
- Success Response: `200 OK` – returns a JSON array of matching notes
- Error Response: `400 Bad Request` (`MISSING_QUERY`) if the `query` parameter is absent or blank; `401 Unauthorized` if the user is not authenticated; `500 Internal Server Error` on database failure

Favorites

- URL: `/rest/favorites/{noteId}` / `/favorites` / `/remove-favorite`
- Method: `POST` (add), `DELETE` (remove via REST), `GET` (page), `POST` (remove via form)
- URL Parameters: `noteId` (int, required for REST endpoints)
- Data Parameters: `/remove-favorite` – `noteId` (int, form parameter)
- Success Response: REST add/delete – `200 OK` with JSON confirmation; `/favorites` – `200 OK`, forwards to `favorites.jsp`; `/remove-favorite` – `302 Found`, redirects to favorites page
- Error Response: `401 Unauthorized` (`USER_NOT_AUTHENTICATED`) if the user is not authenticated; `500 Internal Server Error` (`DATABASE_ERROR`) on database failure

Ratings

- URL: `/rest/ratings/{noteId}`
- Method: `POST` (create or update), `DELETE` (remove), `GET` (retrieve)
- URL Parameters: `noteId` (int, required)
- Data Parameters: `POST` – `value` (int, 1–5)
- Success Response: `200 OK` – returns a JSON object with rating details (note average, user rating, count)
- Error Response: `400 Bad Request` (`MISSING_NOTE_ID`, `INVALID_NOTE_ID`, `INVALID_VALUE`, `INVALID_PARAMETERS`) for malformed input; `401 Unauthorized` (`USER_NOT_AUTHENTICATED`) if the user is not authenticated; `500 Internal Server Error` (`DATABASE_ERROR`) on database failure

Course Summary

- URL: `/rest/courses/summary`
- Method: `GET`
- URL Parameters: None
- Data Parameters: None
- Success Response: `200 OK` – returns a JSON array of course summaries, each containing course name and note count
- Error Response: `401 Unauthorized` if the user is not authenticated; `500 Internal Server Error` on database failure

Luca Dusi He contributed to the initial project setup by adding the Maven configuration, creating the first project structure, and preparing the first login and register pages with the related servlets. He also implemented important application features such as `CourseSummaryServlet`, `DownloadNoteServlet`, `FavoritesPageServlet`, updates to the register flow and `UserDAO`, the profile page, and significant changes to the upload page and JavaScript. For the report, he added the sequence diagram and the wrote the objectives section.

Laszlo Kosa He contributed to the user and note management part of the application by implementing the `User` model, `UserDAO`, and `NoteDAO`, and by adapting the login and registration logic to use them. He also implemented object storage support through `StorageService`, updated the database connection and dependencies, and developed the note upload functionality with its servlet and interface. In addition, he implemented `FavoriteServlet`, refined the registration page, and contributed to the documentation with the main functionalities and REST error codes sections.

Milos Trifunovic He contributed to the core domain and business logic by implementing the `Note`, `Course`, `Rating`, and `Favorite` models, the search notes functionality, the rating servlet, the note update servlet, and further work on `RatingDAO`. He also worked on the frontend by improving the rating and favorite interactions, adding the favorites page, and reorganizing the controller structure. He prepared the database schema, initial SQL data, and the environment configuration template, and contributed heavily to the report with the data logic layer, presentation logic layer, class diagram and screenshots.

Edoardo Zanella He contributed to repository maintenance and application features, and cleaning project files. He implemented `CourseDAO`, `RatingDAO`, `FavoriteDAO`, `LogoutServlet`, `RemoveFavoriteServlet`, `ProfileServlet`, and `DeleteNoteServlet`, and he also created the home page while extending the course data access layer. For the report, he added and refined the REST API summary and detailed REST API documentation.